Pizza Hut Sales

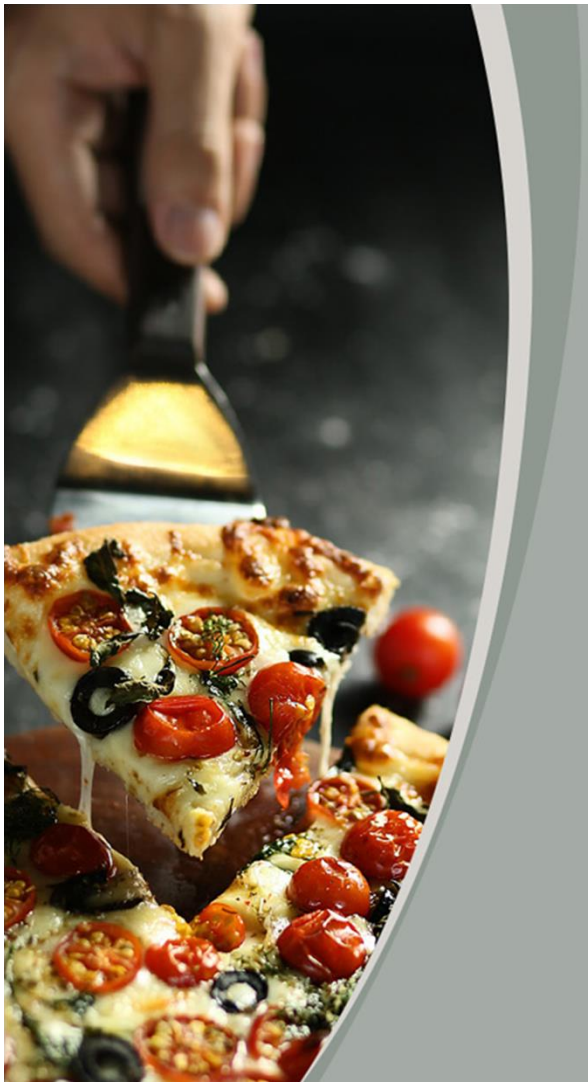
Data Analysis Using SQL

Problem Statement

The objective of this project is to analyze Pizza Hut sales data to extract meaningful business insights. The dataset contains information about orders, pizza types, quantities, pricing, and timestamps.

The goal is to:

- Understand customer ordering behavior
- Identify top-performing products
- Analyze revenue trends
- Support data-driven decision-making for business growth



Objectives

- Calculate total orders and revenue
- Identify best-selling pizzas and categories
- Analyze order patterns by time and date
- Evaluate revenue contribution by category
- Generate insights for business optimization

Tools & Technologies Used

- SQL Server
- T-SQL
- Data Cleaning Techniques
- Aggregations & Window Functions
- Joins & Subqueries

Dataset Description

The dataset consists of the following four tables:

Orders

- order_id
- date
- time

Order Details

- order_details_id
- order_id
- pizza_id
- quantity

Pizzas

- pizza_id
- pizza_type_id
- size
- price

Pizza Type

- pizza_type_id
- name
- category

Data Cleaning Process

Before analysis, the dataset was cleaned to ensure accuracy:

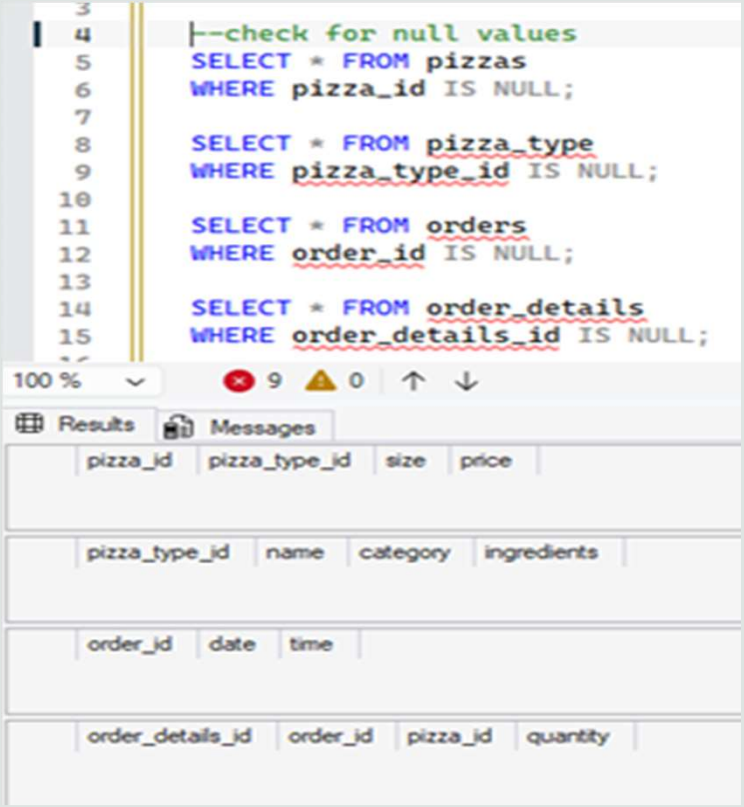
1. Null Value Check

Verified all primary key columns

--check for null values

```
SELECT * FROM pizzas WHERE pizza_id IS NULL;
SELECT * FROM pizza_type WHERE pizza_type_id IS NULL;
SELECT * FROM orders WHERE order_id IS NULL;
SELECT * FROM order_details WHERE order_details_id IS NULL;
```

No null values found in ID columns.



Data Cleaning Process

2. Data Type Correction

- Check if data type is appropriate
- In '**Pizzas**' table converted price column from **VARCHAR → DECIMAL(10,2)**

*-- Verified column structure using **INFORMATION_SCHEMA***

select * from INFORMATION_SCHEMA.COLUMNS where TABLE_NAME = 'pizzas'

select * from pizzas;

-- change data type of price column from varchar to decimal

ALTER TABLE pizzas

ALTER COLUMN price DECIMAL(10,2);

```
29
30  -- change data type of price column fron varchar to decimal
31
32  ALTER TABLE pizzas
33  ALTER COLUMN price DECIMAL(10,2);
```

00 % 9 0

Messages

```
Commands completed successfully.

Completion time: 2026-04-29T20:02:34.8723932+05:30
```

```
16
17  --Check if data type is appropriate
18  select * from INFORMATION_SCHEMA.COLUMNS where TABLE_NAME = 'pizzas'
19  select * from pizzas
20
21
```

100 % 9 0 Ln: 17, Ch: 1 (130 chars, 3 lines) SPC CRLF UT

Results Messages

	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	COLUMN_DEFAULT	IS_NULLABLE	DATA_TYPE	CHARACTER_MAXIMUM_LE
1	Pizzahut	dbo	pizzas	pizza_id	1	NULL	YES	varchar	50
2	Pizzahut	dbo	pizzas	pizza_type_id	2	NULL	YES	varchar	50
3	Pizzahut	dbo	pizzas	size	3	NULL	YES	varchar	50
4	Pizzahut	dbo	pizzas	price	4	NULL	YES	decimal	NULL

	pizza_id	pizza_type_id	size	price
1	bbq_ckn_s	bbq_ckn	S	12.75
2	bbq_ckn_m	bbq_ckn	M	16.75
3	bbq_ckn_l	bbq_ckn	L	20.75
4	cali_ckn_s	cali_ckn	S	12.75
5	cali_ckn_m	cali_ckn	M	16.75

Activate Windows

Data Cleaning Process

2. Data Type Correction

Check '**Orders**' table and set appropriate data type for **date** and **time** column.

```
-- Verified column structure using INFORMATION_SCHEMA
select * from INFORMATION_SCHEMA.COLUMNS where TABLE_NAME = 'orders'
select * from Orders

--Alter data type for  date and time column
ALTER TABLE orders ALTER COLUMN date date
ALTER TABLE orders ALTER COLUMN time time;
```

In '**Orders**' table converted:

- **Date** column from **VARCHAR → DATE**
- **Time** column from **VARCHAR → TIME**

```
select * from INFORMATION_SCHEMA.COLUMNS where TABLE_NAME = 'orders'
select * from dbo.Orders
```

100 % | 1 | 0 | Ln: 4, Ch: 16 | TABS | CRLF | UTF-8

Results | Messages

	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	COLUMN_DEFAULT	IS_NULLABLE	DATA_TYPE	CHARACTER_MAXIMUM_LE
1	Pizzahut	dbo	orders	order_id	1	NULL	YES	varchar	50
2	Pizzahut	dbo	orders	date	2	NULL	YES	varchar	50
3	Pizzahut	dbo	orders	time	3	NULL	YES	varchar	50

```
select * from INFORMATION_SCHEMA.COLUMNS where TABLE_NAME = 'orders'
select * from dbo.Orders

ALTER TABLE orders
ALTER COLUMN date date
ALTER TABLE orders
ALTER COLUMN time time;
```

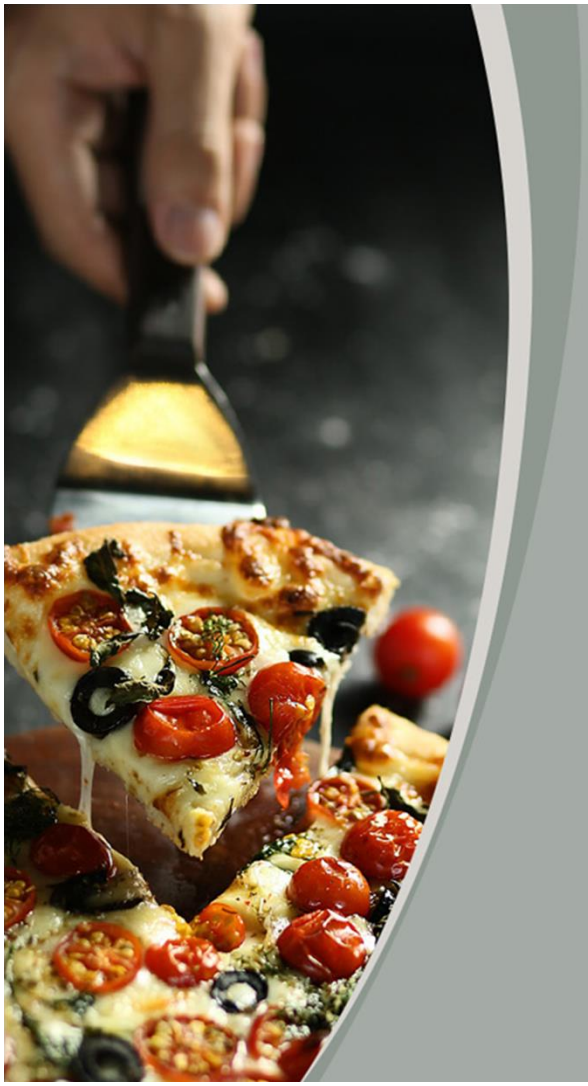
100 % | 1 | 0 | Ln: 7, Ch: 1 | TABS | CRLF | UTF

Results | Messages

	TABLE_CATALOG	TABLE_SCHEMA	TABLE_NAME	COLUMN_NAME	ORDINAL_POSITION	COLUMN_DEFAULT	IS_NULLABLE	DATA_TYPE	CHARACTER_MAXIMUM_LE
1	Pizzahut	dbo	orders	order_id	1	NULL	YES	varchar	50
2	Pizzahut	dbo	orders	date	2	NULL	YES	date	NULL
3	Pizzahut	dbo	orders	time	3	NULL	YES	time	NULL

	order_id	date	time
1	1	2015-01-01	11:38:36.0000000
2	2	2015-01-01	11:57:40.0000000
3	3	2015-01-01	12:12:28.0000000
4	4	2015-01-01	12:16:31.0000000
5	5	2015-01-01	12:21:30.0000000
6	6	2015-01-01	12:29:36.0000000

Activate Windows



Business Impact

- Helps identify high-demand products
- Supports pricing and promotion strategies
- Improves operational efficiency
- Enables data-driven decision making

Pizza Sales Analysis Using SQL

Basic Analysis

- Total number of orders
- Total revenue generated
- Highest-priced pizza
- Most common pizza size
- Top 5 most ordered pizzas
- Total quantity sold by category
- Category-wise pizza distribution

Intermediate to Advanced Analysis

- Orders distribution by hour
- Average pizzas ordered per day
- Top 3 pizzas based on revenue
- Revenue contribution (%) by category
- Cumulative revenue over time
- Top 3 pizzas per category (using ranking)

Pizza Sales Analysis Using SQL

1. Retrieve the total number of orders placed.

SQL QUERIES

--Retrieve the total number of orders placed.

--select count(*) from dbo.orders;

select count(order_id) as total_orders from dbo.orders;

```
--Retrieve the total number of orders placed.
--select count(*) from dbo.orders;
select count(order_id) as total_orders from dbo.orders;
```

100 % No issues found

Results | Messages

	total_orders
1	21350

The total number of orders placed are **21350.**

Pizza Sales Analysis Using SQL

2. Calculate the total revenue generated from pizza sales.

SQL QUERIES

--Calculate the total revenue generated from pizza sales.

```
select round(sum(o.quantity * p.price),2) as total_revenue
from order_details o  join pizzas p on o.pizza_id = p.pizza_id  ;
```

```
--Calculate the total revenue generated from pizza sales.
select round(sum(o.quantity * p.price),2) as total_revenue
from order_details o  join pizzas p on o.pizza_id = p.pizza_id  ;
```

100 % No issues found

Results | Messages

	total_revenue
1	817860.05

The **total revenue** generated from pizza sales is **₹817860.05**

Pizza Sales Analysis Using SQL

3. Identify the highest-priced pizza.

SQL QUERIES

--Identify the highest-priced pizza.

```
select top 1
name ,
price
from pizza_type pt
inner join pizzas p
on pt.pizza_type_id = p.pizza_type_id
order by price desc
```

```
--3.    Identify the highest-priced pizza.
select top 1 name , price from pizza_type pt inner join pizzas p
on pt.pizza_type_id = p.pizza_type_id
order by price desc
```

100 % No issues found

Results | Messages

	name	price
1	The Greek Pizza	35.95

The highest-priced pizza is ‘**The Greek Pizza**’ with price **₹35.95**

Pizza Sales Analysis Using SQL

4. Identify the most common pizza size ordered.

SQL QUERIES

```
--Identify the most common pizza size ordered.

select p.size, count(od.order_details_id)
order_countfrom pizzas p
inner join order_details od
on od.pizza_id = p.pizza_id
group by p.size
order by order_count desc
```

The most common sized is '**Large**' sized pizza with **18526** orders.

```
--4.    Identify the most common pizza size ordered.
select p.size, count(od.order_details_id) order_count
from pizzas p
inner join order_details od on od.pizza_id = p.pizza_id

group by p.size
order by order_count desc
```

100 % No issues found

Results | Messages

	size	order_count
1	L	18526
2	M	15385
3	S	14137
4	XL	544
5	XXL	28

Pizza Sales Analysis Using SQL

5. List the top 5 most ordered pizza types along with their quantities.

SQL QUERIES

```sql
--List the top 5 most ordered pizza
types along with their quantities.

select top 5 pt.name ,
sum(od.quantity) total_quantity

from pizza_type pt
inner join pizzas p
on pt.pizza_type_id = p.pizza_type_id
inner join order_details od on
od.pizza_id = p.pizza_id

group by pt.name

order by total_quantity desc
```

```sql
--5.    List the top 5 most ordered pizza types along with their quantities.
select top 5 pt.name , sum(od.quantity) total_quantity
from pizza_type pt
inner join pizzas p on pt.pizza_type_id = p.pizza_type_id
inner join order_details od on od.pizza_id = p.pizza_id
group by pt.name
order by total_quantity desc
```

The **top most ordered pizza** type are:

	name	total_quantity
1	The Classic Deluxe Pizza	2453
2	The Barbecue Chicken Pizza	2432
3	The Hawaiian Pizza	2422
4	The Pepperoni Pizza	2418
5	The Thai Chicken Pizza	2371

Pizza Sales Analysis Using SQL

6. Join the necessary tables to find the total quantity of each pizza category ordered.

SQL QUERIES

```
--Join the necessary tables to find the
total quantity of each pizza category
ordered.

select pt.category ,
sum(od.quantity) total_pizza_qty
from Pizza_type pt
inner join pizzas p
on pt.pizza_type_id = p.pizza_type_id
inner join order_details od
on od.pizza_id = p.pizza_id

group by pt.category
order by total_pizza_qty desc
```

The **total quantity** of each **pizza category** ordered:

Results | Messages

	category	total_pizza_qty
1	Classic	13529
2	Supreme	11987
3	Veggie	11649
4	Chicken	11050
5	Mushroom	1359

Pizza Sales Analysis Using SQL

7. Determine the distribution of orders by hour of the day.

SQL QUERIES

```sql
--Determine the distribution of orders
by hour of the day.

select
datepart(hour,o.time) hour_value,
count(o.order_id) as order_count

from orders o

group by datepart(hour,o.time)

order by hour_value
```

The **distribution of orders by hour** of the day.

Results | Messages

	hour_value	order_count
1	9	1
2	10	8
3	11	1231
4	12	2520
5	13	2455
6	14	1472
7	15	1468
8	16	1920
9	17	2336
10	18	2399
11	19	2009

Pizza Sales Analysis Using SQL

9. Group the orders by date and calculate the average number of pizzas ordered per day.

SQL QUERIES

```sql
--Group the orders by date and calculate the average number of pizzas ordered per day.

select
avg(cast(total_qty as decimal(10,2)))
average
from (
select o.date, sum(od.quantity) total_qty
from orders o
inner join order_details od
on o.order_id = od.order_id
group by o.date
) as total_qty_per_day
```

```sql
--9.    Group the orders by date and calculate the averag

select avg(cast(total_qty as decimal(10,2))) average
from
(select o.date, sum(od.quantity) total_qty
from orders o
inner join order_details od on o.order_id = od.order_id
group by o.date) as total_qty_per_day
```

0 % No issues found

Results | Messages

average
138.47

The **average number of pizzas ordered per day** is 138.47

Pizza Sales Analysis Using SQL

10. Determine the top 3 most ordered pizza based on revenue.

SQL QUERIES

```
--Determine the top 3 most ordered pizza
based on revenue.

select top 3 pt.name ,
sum(od.quantity * p.price) as revenue

from pizzas p
inner join order_details od
on p.pizza_id = od.pizza_id

inner join Pizza_type pt
on pt.pizza_type_id = p.pizza_type_id

group by pt.name
order by revenue desc
```

Results | Messages

	name	revenue
1	The Thai Chicken Pizza	43434.25
2	The Barbecue Chicken Pizza	42768.00
3	The California Chicken Pizza	41409.50

The **top 3 most ordered pizza based on revenue** are:

1. The Thai Chicken Pizza
2. The Barbecue Chicken Pizza
3. The California Chicken Pizza

Pizza Sales Analysis Using SQL

11. Calculate the percentage contribution of each pizza type to total revenue.

SQL QUERIES

```
select t.category,  t.revenue,
CAST(revenue * 100.0 / total_revenue AS DECIMAL(10,2))
AS contribution_percent
from(
    select
    pt.category ,
    sum(od.quantity * p.price) as revenue,
    sum(sum(od.quantity * p.price)) over() as
    total_revenue
    from pizzas p
    inner join order_details od
    on p.pizza_id = od.pizza_id
    inner join Pizza_type pt
    on pt.pizza_type_id = p.pizza_type_id
    group by pt.category
    ) t
    order by revenue desc
```

The **percentage contribution** of each pizza type to total revenue:

Results | Messages

	category	revenue	contribution_percent
1	Supreme	208197.00	25.46
2	Classic	201218.60	24.60
3	Chicken	195919.50	23.96
4	Veggie	193690.45	23.68
5	Mushroom	18834.50	2.30

Pizza Sales Analysis Using SQL

12. Analyze the cumulative revenue generated over time.

SQL QUERIES

```sql
-- Analyze the cumulative revenue generated over time.

select t.date,t.revenue,
sum(t.revenue) over(order by t.date) as
cumm_revenue
from
   (select o.date,
   sum( od.quantity * p.price) as revenue
   from orders o
   inner join order_details od on od.order_id =
   o.order_id
   inner join pizzas p on p.pizza_id =
   od.pizza_id
   group by o.date
   ) t
```

Below is the **cumulative revenue generated over time** :

Results | Messages

	date	revenue	cumm_revenue
4	2015-01-04	1755.45	9863.60
5	2015-01-05	2065.95	11929.55
6	2015-01-06	2428.95	14358.50
7	2015-01-07	2202.20	16560.70
8	2015-01-08	2838.35	19399.05
9	2015-01-09	2127.35	21526.40

Query executed successfully.

Pizza Sales Analysis Using SQL

13. Determine the top 3 most ordered pizza types based on revenue for each pizza category.

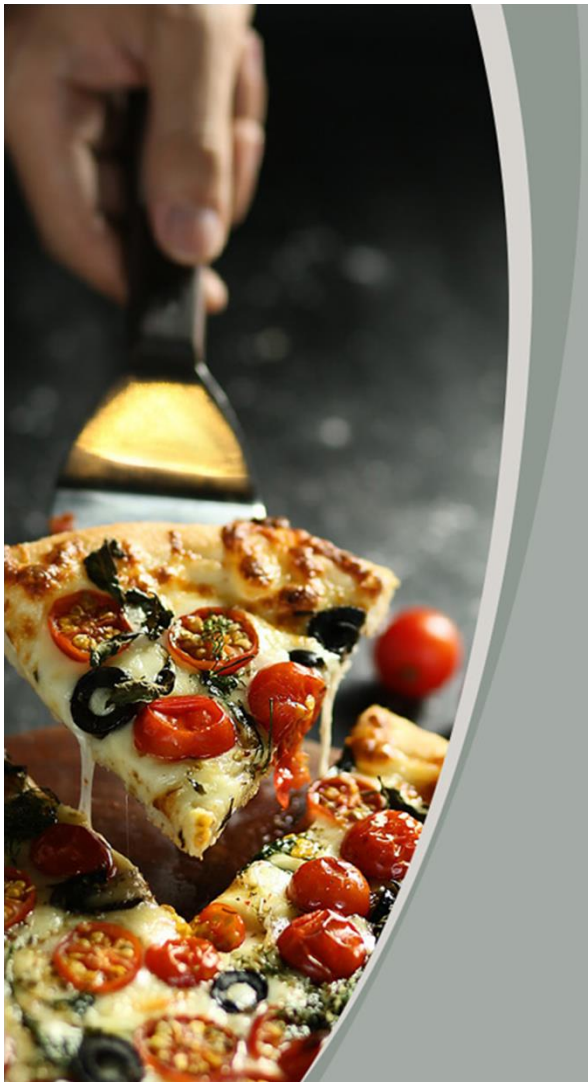
SQL QUERIES

```
select * from
(select t.name,t.category ,t.revenue,
rank() over(partition by category order by revenue desc)
as revenue_by_each_category
from(
  select pt.name, pt.category,
  sum(od.quantity * p.price) as revenue
  from orders o
  inner join order_details od on od.order_id = o.order_id
  inner join pizzas p on p.pizza_id = od.pizza_id
  inner join Pizza_type pt on pt.pizza_type_id =
p.pizza_type_id
  group by pt.name, pt.category
  )t
) t1
where revenue_by_each_category <= 3
```

Below are **the top 3 most ordered pizza types based on revenue for each pizza category**

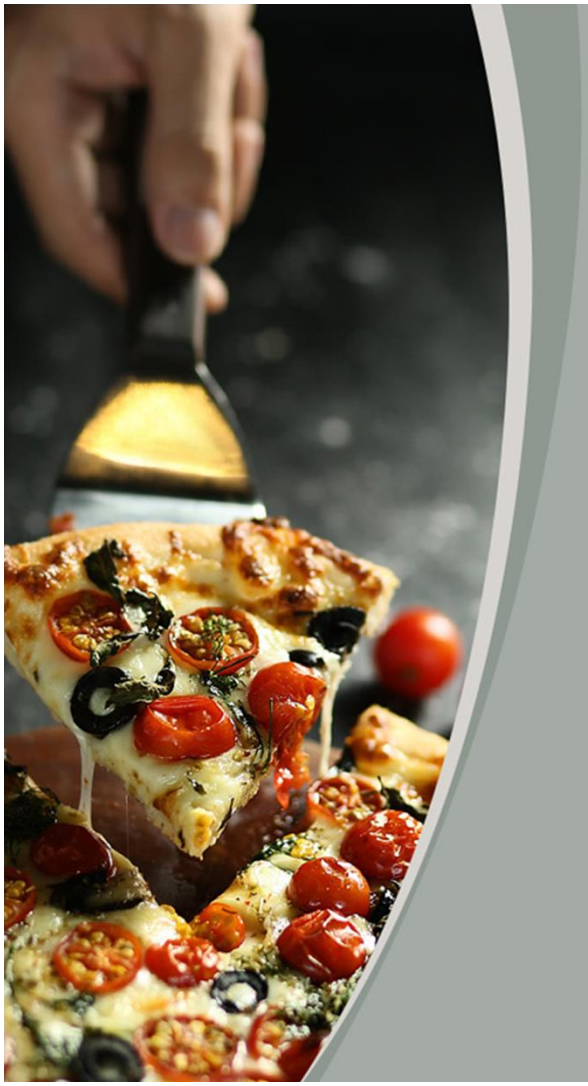
Results | Messages

	name	category	revenue	revenue_by_each_category
1	"The Pepperoni	Mushroom	18834.50	1
2	The Thai Chicken Pizza	Chicken	43434.25	1
3	The Barbecue Chicken Pizza	Chicken	42768.00	2
4	The California Chicken Pizza	Chicken	41409.50	3
5	The Classic Deluxe Pizza	Classic	38180.50	1
6	The Hawaiian Pizza	Classic	32273.25	2
7	The Pepperoni Pizza	Classic	30161.75	3
8	The Spicy Italian Pizza	Supreme	34831.25	1
9	The Italian Supreme Pizza	Supreme	33476.75	2
10	The Sicilian Pizza	Supreme	30940.50	3



Conclusion

- This project demonstrates how SQL can be used to transform raw transactional data into actionable insights.
- By analyzing sales trends, customer preferences, and revenue distribution, businesses can make informed strategic decisions.



Future Enhancements

- Build Power BI dashboard for visualization
- Add customer segmentation analysis
- Forecast future sales using time-series models
- Integrate real-time data pipelines

Thank you!